



中国科学院大学

University of Chinese Academy of Sciences

研究生学位论文开题报告

报告题目 单点登录系统的实现安全性分析

学生姓名 朱正轩 学号 2023E8015082070

指导教师 徐静 职称 研究员

学位类别 电子信息硕士

学科专业 软件工程

研究方向 网络空间安全

培养单位 中国科学院软件研究所

填表日期 2025 年 06 月

中国科学院大学制

报告提纲

一 选题的背景及意义	1
1.1 选题背景	1
1.2 选题意义	5
二 国内外本学科领域的发展现状与趋势	7
2.1 OAuth 实现中的常见漏洞	7
2.2 SSO 测量工具	10
三 课题主要研究内容、预期目标	11
3.1 研究内容	11
3.2 预期目标	11
四 拟采用的研究方法、技术路线、实验方案及其可行性分析	12
4.1 研究方法	12
4.2 技术路线和方案	12
4.3 可行性分析	13
五 已有科研基础与所需的科研条件	14
5.1 已有的科研基础	14
5.2 所需的科研条件	14
六 研究工作计划与进度安排	15

一 选题的背景及意义

1.1 选题背景

在互联网早期，用户访问一个网站，就需要注册一个账号，设置一个密码。随着在线服务的迅猛发展，越来越多的网站、应用、企业系统相继上线，人们不得不为每一个服务单独创建账号。工作用邮箱一个密码，报销系统一个密码，文件共享平台又是另一个密码；而在个人生活中，从社交媒体到购物平台，几乎每个地方都要“注册-登录”这一套流程。

很快，问题显现出来了：用户要么设置一堆容易忘记的复杂密码，要么干脆多个系统共用同一个弱密码，安全性大打折扣。频繁地登录、切换账号也让体验变得烦琐低效。对于企业管理员来说，管理成千上万个用户账号、处理重置密码请求、应对权限分散等问题也变得日益繁重。

在这样的背景下，单点登录（Single Sign-On, SSO）应运而生。它的目标很明确：只登录一次，畅通所有服务。通过建立一个统一的身份认证中心，用户只需输入一次用户名和密码，系统就会在后台完成对多个相关系统的身份传递，从而实现一次登录，多点通行。

单点登录是一种身份验证方法，它简化了用户的认证流程，允许用户使用一组凭证登录多个应用程序和网站。当用户登录一个应用程序时，会自动登录到其他相关联的应用程序，无论它们使用的是哪个域、平台或技术。这减轻了用户在不同账户和服务中管理多个用户名和密码的负担。例如，当用户登录谷歌时，其凭证会自动在相关联的服务（如 Gmail 和 YouTube）中得到验证，而无需分别登录每个服务。

本研究致力于提出一个针对单点登录系统的实现漏洞的通用性研究方案，尝试发现针对单点登录系统实现的新的攻击方法和解决方案，提高单点登录认证的安全性。此外开发 SSO 系统测量工具，对目前网站上应用的单点登录认证流程的安全性进行评估，为网络应用开发者提供身份认证安全性的检测方法。

1.1.1 单点登录

单点登录是一种联合身份管理（FIM）工具，在身份和访问管理（IAM）中执行关键流程——身份验证，这是一种框架，允许组织在用户和设备进入网络时安全地确认其身份，这对于分配用户访问权限以及确保用户仅具有有效履行其职责所需的适当访问级别至关重要。

单点登录 (SSO) 通过在服务提供方和服务身份提供方之间共享和验证登录凭证来实现。服务提供方 (SP) 通常是向用户和组织提供产品、解决方案和服务的供应商, 例如应用程序或网站。身份提供方 (IdP) 是一个创建、管理和维护用户身份, 并提供身份验证服务以验证用户的系统。这些受信任的提供方使用户能够使用单点登录访问应用程序和网站, 并通过减少密码疲劳来改善用户体验。

SSO 分为两种: a) 同域 SSO 当在一个企业域名下, 可以通过二级域名区分不同的系统, 实现 SSO。例如: app1.a.com 和 app2.a.com 作为应用系统, 以及 sso.a.com 登录系统, 只要在 sso.a.com 登录, app1.a.com 和 app2.a.com 就也登录了。根据登录认证机制, 在 sso.a.com 中登录了, 在 sso.a.com 的服务端的 session 中记录了登录状态, 同时在浏览器端 (Browser) 的 sso.a.com 下写入了 cookie。此时会出现两个问题: cookie 不能跨域, session 不共享。sso 登录以后, 可以将 cookie 的域设置为顶域, 即.a.com, 这样所有子域的系统都可以访问到顶域的 cookie。我们在 sso 系统登录状态, 这时再访问 app1, cookie 也带到了 app1 的服务 (Server)。这里需要把两个 app 系统的 session 共享, 共享 session 的解决方案有很多, 例如: Spring-Session。通过巧用 cookie 顶域的特性和 session 共享, 可以实现同域下的多个系统登录认证, 也就是同域下的单点登录 b) 跨域 SSO 跨域单点登录 (Cross-domain SSO) 指的是在多个不同的顶级域名之间实现单次登录即可访问多个系统。例如, a.com 登录后, 可以直接访问 b.com 的相关资源, 无需再次输入凭据。跨域 SSO 的安全核心在于 Token 传递的安全性, 如果实现不当, 可能带来 Token 窃取、重放攻击、CSRF 等问题, 因此需要严格的安全策略。如今, 最广泛采用的单点登录 (SSO) 协议是 SAML、OAuth 2.0 和 OpenID Connect 1.0, 本研究关注于后两种协议。

1.1.2 OAuth2.0 协议

OAuth 2.0 是一种授权框架 [1], 允许第三方应用在用户授权下访问其存储在服务提供者上的资源, 而无需共享用户凭证。用于不信任第三方应用时, 允许用户授权第三方应用访问他们存储在另外的服务提供者上的信息, 而不需要将用户名和密码提供给第三方应用或分享他们数据的所有内容。OAuth 2.0 执行过程如下:

1. 用户访问第三方应用程序 (简称: 客户端) 以后, 客户端要求用户给予授权。
2. 用户同意给予客户端授权。
3. 客户端使用第 2 步获得的授权, 向认证服务器申请令牌。

4. 认证服务器对客户端进行认证以后，确认无误，同意发放令牌。
5. 客户端使用令牌，向资源服务器申请获取资源。
6. 资源服务器确认令牌无误，同意向客户端开放资源。

上述中的第 2 步是关键，即用户如何给予客户端授权。有了该授权后，客户端就可以获取令牌，进而凭令牌获取资源。

授权模式: OAuth 2.0 定义了四种授权方式：

1. 授权码模式 (authorization code)：第三方应用先申请一个授权码，然后再用该码获取令牌。

2. 简化模式 (implicit)：有些 Web 应用是纯前端应用，没有后端，这时必须将令牌储存在前端。RFC 6749 规定了第二种方式，允许直接向前端颁发令牌。这种方式没有授权码这个中间步骤，所以称为（授权码）“隐藏式” (implicit)。

3. 密码模式 (resource owner password credentials)：如果高度信任某个应用，RFC 6749 也允许用户把用户名和密码，直接告诉该应用。该应用使用密码，申请令牌，这种方式称为“密码式” (password)。

4. 客户端模式 (client credentials)：适用于没有前端的命令行应用，即在命令行下请求令牌。

授权码模式: 授权码模式是功能最完整、流程最严密安全的授权模式。它的特点就是通过客户端的后台服务器，与“服务提供商”的认证服务器进行互动。其执行流程如下：

1. 用户访问客户端，客户端将用户导向认证服务器。
2. 用户选择是否给予客户端授权。
3. 假设用户给予授权，认证服务器将用户导向客户端事先指定的“重定向 URI” (redirection URI)，同时附上一个授权码（每个用户的授权码不同）。
4. 客户端收到授权码，附上早先的“重定向 URI”，向认证服务器申请令牌。这一步是在客户端的后台服务器上完成，对用户不可见。
5. 认证服务器核对了授权码和重定向 URI，确认无误后，向客户端发送访问令牌 (access token) 和更新令牌 (refresh token)。

从上述流程可知，只有第 2 步需要用户进行授权操作，之后的流程都是在客户端的后台和认证服务器后台之前进行“静默”操作，对于用户来说是无感知的。

1.1.3 OpenID Connect 1.0 协议

OpenID Connect 发布于 2014 年，是建立在 OAuth 2.0 协议之上的简单身份层，它允许客户端基于授权服务器或身份提供商（IdP）进行的身份验证来验证最终用户的身份，并获得用户的相关信息。与单纯的 OAuth2.0 不同，OIDC 不仅关注于授权（即允许应用程序访问用户在其他服务上的资源），更强调身份验证——确认“你是谁”。OpenID Connect 提供了 RESTful HTTP API，并使用 Json 作为数据的传递格式。相比于基于 XML 格式的 SAML 协议，OpenID Connect 因为其更加简洁的数据交换格式，被越来越多的应用使用，已经成为事实上的标准。

OIDC 引入了 ID Token 的概念，这是一种包含用户身份信息的 JWT（JSON Web Token），使得应用可以确信“谁”正在访问，而不仅仅是可以访问什么。

如下图1取自 Core 规范文档 [2]，OIDC 要做的事情就是用户通过认证服务器来证明自己的身份，从而访问三方服务，所以 OIDC 规范规定协议参与方包含如下 3 个角色：EU（End User），即用户；OP（OpenID Provider），即提供身份证明的认证服务器；RP（Relying Party），即验证身份的第三方服务。协议流程中 AuthN=Authentication，表示认证；AuthZ=Authorization，代表授权。注意这里 RP 发往 OP 的请求，是属于 Authentication 类型的请求，虽然在 OIDC 中是复用 OAuth2 的 Authorization 请求通道，但是用途是不一样的，且 OIDC 的 AuthN 请求中 scope 参数必须要有一个值为 openid 的参数，用来区分这是一个 OIDC 的 Authentication 请求，而不是 OAuth2 的 Authorization 请求。其执行流程如下：

1. 用户请求访问 RP：用户尝试访问依赖方（RP）提供的受保护资源。
2. 重定向至 IdP：RP 将用户重定向到预先配置的身份提供商（IdP）进行登录。
3. 用户身份验证：用户在 IdP 上输入凭证完成身份验证。
4. 授权码发放：IdP 向用户代理（通常是浏览器）返回一个授权码，并附带 RP 的重定向 URI。
5. RP 交换令牌：RP 通过后端服务器向 IdP 发送授权码，请求换取访问令牌和 ID 令牌。
6. 验证 ID 令牌：RP 验证 ID 令牌的有效性（签名、过期时间等），并提取用户信息。
7. 访问资源：验证成功后，RP 允许用户访问受保护资源。

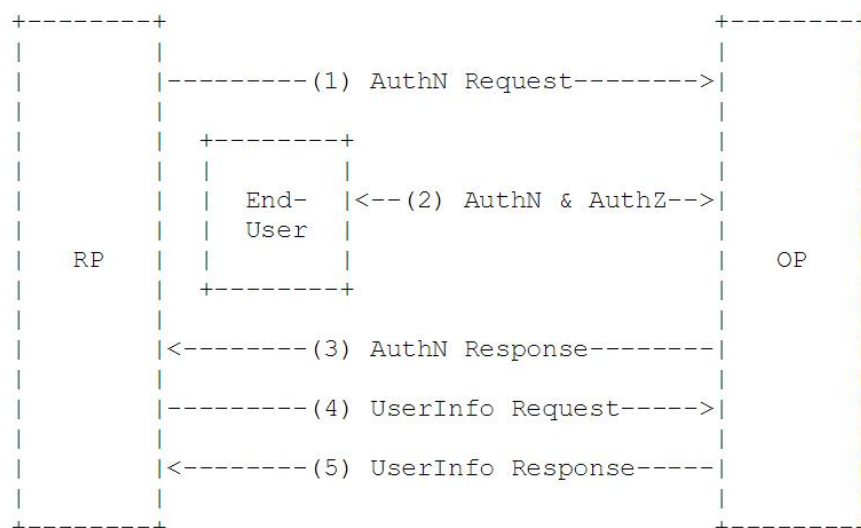


图 1 OpenID connect 1.0

1.2 选题意义

1.2.1 OAuth 协议普及性

OAuth 2.0 协议是部署最广泛的授权/单点登录（SSO）协议之一，同时也是新 SSO 标准 OpenID Connect 的基础。OAuth 在众多在线服务中已广泛流行，成为现代网络安全基础设施的重要组成部分。该协议的采用体现在第三方应用程序与热门在线平台的无缝集成上，使用户能够在不泄露凭证的情况下授予对其资源的有限访问权限。OAuth 的这种广泛应用突显了其在数字领域确保安全和标准化授权机制方面的重要性。

在电子商务领域，OAuth 被在线零售平台广泛采用，以促进用户与第三方应用程序之间的安全交易和互动。该协议在这一领域的普及凸显了其在增强用户信任和数据安全方面的作用，从而为在线购物体验营造了有利环境。此外，OAuth 在社交媒体平台中的整合彻底改变了用户互动方式，实现了信息的无缝共享，同时保持严格的安保措施。

教育领域也见证了 OAuth 在实现安全访问在线学习资源方面的采用率大幅上升。平台和资源。通过实施 OAuth 协议，教育机构能够确保学生和教职员工安全地访问课程材料，并在数字平台上进行协作，同时不会泄露敏感数据。此外，医疗保健行业也采用了 OAuth 来保护患者信息，并实现跨各种在线医疗保健服务的安全访问 [3]。

1.2.2 OAuth 实现漏洞

尽管 OAuth 已被广泛用作在线服务中安全授权的标准协议，但具体实现中仍然存在漏洞，给用户数据的安全和隐私保护带来了重大挑战。各种研究都强调了 OAuth 实现中存在的漏洞的普遍性，这些漏洞涵盖了从常见的网络应用程序漏洞（如跨站请求伪造攻击）到更为复杂的攻击手段等各类情况，许多攻击利用了协议设计和实现中的漏洞 [4–6]。

在 OAuth 实施过程中所发现的关键问题之一是未能遵循推荐的安全措施和准则，这使得系统容易受到攻击，可能损害用户账户和泄露敏感信息 [5]。现代网络生态系统的复杂性，包括物联网（IoT）和智能城市应用，也引入了新的攻击途径，这些途径利用了 OAuth 实现中的漏洞，从而可能导致潜在的安全风险 [6, 7]。

此外，OAuth 2.0 生态系统中高曝光度的攻击事件和实现漏洞的发现引发了人们对依赖 OAuth 进行授权的在线服务整体安全态势的担忧 [8, 9]。这些漏洞包括代码注入、CSRF、XSS 和授权码泄露等问题，凸显了需要采取强有力的安全措施来降低与 OAuth 实现相关的风险 [10]。

此外，开发人员和用户对 OAuth 漏洞的缺乏认识，再加上网络威胁日益复杂化，进一步加剧了在线平台所面临的安全挑战 [11]。OAuth 系统的实际应用常常遭受 CSRF 攻击和其他安全漏洞的侵害，这凸显了需要持续监控和改进 OAuth 安全实践的必要性 [11]。

1.2.3 安全的 OAuth 实现的重要性

确保 OAuth 的安全实施对于保护在线服务中的用户数据至关重要。在部署 OAuth 协议时遵循最佳实践和严格的安全措施，服务器能够降低未经授权访问、数据泄露和隐私侵犯所带来的风险。安全地实施 OAuth 不仅能保护敏感的用户信息，还能增强用户之间的信任，从而提升在线平台的整体用户体验和信誉度 [12]。

安全的 OAuth 实现对于防止可能危及用户数据的安全漏洞起着至关重要的作用。通过进行全面的安全评估和风险分析，开发者能够识别并解决 OAuth 实现中的薄弱环节，从而降低恶意行为者利用这些漏洞的可能性 [12, 13]。实施额外的安全层，如多因素身份验证和加密，将进一步加强通过 OAuth 协议传输的用户数据的安全保护 [14, 15]。

此外，安全的 OAuth 实现对于遵守数据保护法规和标准至关重要。通过遵循安全最佳实践和指南，组织能够表明其致力于保护用户隐私和机密性，从而避免因数据泄露

而产生的法律后果和财务处罚 [16]。安全的 OAuth 实现还符合安全透明度的原则，使用户能够更好地控制其数据和隐私设置 [16, 17]。

总之，OAuth 的安全实现对于保护用户数据、维持对在线服务的信任以及确保符合数据保护法规至关重要。通过优先考虑安全措施、定期进行安全审计以及及时了解新的威胁，组织能够增强其 OAuth 实施的抗风险能力，并在日益互联的数字环境中保护用户数据。

作为多方参与的协议，OAuth 的安全性需要资源所有者、客户端应用程序、授权认证服务器来共同保证。特别是，OAuth 服务器实现中的漏洞可能会导致严重的后果，因为这会影响到所有与服务提供商合作的客户端应用程序（以及这些应用程序的所有资源所有者）。例如，2019 年在微软 OAuth 服务器的重定向 URI 验证机制中发现的一个漏洞使得攻击者能够接管微软 Azure 账户 [18]。此外，在 2018 年，Facebook OAuth 服务器的一个漏洞使得攻击者能够窃取近 5000 万用户由授权服务器颁发的访问令牌 [19]。

即便是像微软和脸书这样的知名服务提供商，在其授权服务器的实施过程中仍会出现严重的错误——这就引出了一个紧迫的问题：如何对 OAuth 服务提供商的实施情况进行安全审查？由于 OAuth 在用户安全和隐私方面具有很高的重要性和显著的影响，此前已经有许多研究工作都集中在 OAuth 的安全性上。然而，据我们所知，目前尚无针对国内 OAuth 协议实现安全性的全面调查，以及缺乏重复可用的能够自动识别服务提供商实现中涉及安全敏感问题的逻辑缺陷的工具。

二 国内外本学科领域的发展现状与趋势

一些研究已经探索了 OAuth 安全漏洞，强调了与 OAuth 实现相关的潜在风险和弱点，对 OAuth 2.0 进行了全面的正式安全分析，指出了协议设计和实现中的潜在漏洞和安全漏洞，揭示了网站授权中已知的和新的漏洞，强调了在 OAuth 实现中健壮的安全措施的重要性，和持续评估和增强 OAuth 安全实践的迫切需要 [4–6, 11, 20–24]。

2.1 OAuth 实现中的常见漏洞

2.1.1 CSRF

OAuth 实现容易受到各种常见漏洞的影响，这些漏洞可能危及用户数据的安全性和完整性。一个普遍的漏洞是跨站点请求伪造 (CSRF)，它允许攻击者欺骗用户在他们

经过身份验证的受信任网站上无意地执行恶意操作 [4, 22]。CSRF 攻击利用用户和服务提供者之间已建立的信任关系，对 OAuth 系统构成重大威胁。

在此基础上，arshad 也研究了针对 OAuth 协议登录前进行的 login CSRF 攻击 [20]，在登录跨站请求伪造中，攻击者欺骗受害者向目标网站的登录端点执行跨站请求。攻击者使用自己的凭据伪造登录请求。如果攻击成功，服务器将向受害者的浏览器发出会话 cookie。结果，受害者使用攻击者的帐户登录到目标网站。乍一看，这种攻击似乎是无害的。通常，跨站点请求伪造攻击涉及对受害者受保护资源执行的操作。在 login CSRF 中，攻击者利用应用程序漏洞欺骗用户在攻击者的帐户内执行一些意外操作。浏览器状态在攻击执行后被改变，受害者可能完全不知道他们正在使用别人拥有的帐户。因此，他们可能会上传敏感文件，与恶意用户共享信用卡号码和其他个人数据 [23, 25–27]。

造成 OAuth CSRF 攻击的主要原因在于客户端未使用或未验证 state 参数，缺乏对请求-响应之间的一致性校验；此外，授权码与用户会话之间的绑定机制薄弱，也使攻击者能够借由重放授权结果干扰用户上下文。Shernan 等在其实证研究中发现，大量 OAuth 实现存在未校验 state 参数的问题 [23]。

2.1.2 授权码拦截

授权码拦截攻击（Authorization Code Interception Attack）是 OAuth 2.0 授权码流程中的典型攻击方式，攻击者通过监听或操纵 redirect_uri 的接收路径，获取授权服务器发放的授权码（authorization code），并在随后步骤中伪装客户端完成令牌交换，从而获取用户访问令牌。该类攻击主要源于客户端与授权码之间缺乏绑定机制，特别是在公共客户端（如 SPA、移动应用）中尤为高发。

该攻击首次被系统性分析并形式化建模的是 [5]，他们在 CCS 发表的工作中指出，OAuth 协议标准本身未要求将授权码与客户端身份、设备环境、会话状态等关键元素绑定，导致攻击者可在代码转发路径中实现拦截。Li 和 Mitchell 也在实证分析中发现，部分 OAuth 实现允许攻击者通过构造中间跳转页面或利用 open redirect，将授权码导流至恶意服务端 [25]。

为应对这一攻击向量，OAuth 社区提出了 PKCE（Proof Key for Code Exchange）机制，最初用于移动端保护，后被广泛扩展至所有公共客户端 [28]。PKCE 通过引入 code_challenge 和 code_verifier 参数，在客户端与授权码之间建立绑定，从而有效防止拦截与重放。此外，OAuth 2.1 草案中明确建议服务端拒绝未使用 PKCE 的授权请求，并

在认证响应中引入 binding context，以强化令牌使用约束 [29]。

尽管 PKCE 有效缓解了大多数拦截场景，但研究者仍指出，其实际部署存在错误配置、fallback 兼容模式及缓存泄露等实现缺陷 [21]，表明授权码保护在实际系统中仍需持续审计与强化。

2.1.3 重定向 URI 攻击

重定向 URI 攻击也是 OAuth 安全研究中的核心问题之一，已被多项实证研究揭示为 OAuth 实现中最普遍的脆弱点之一。Fett 在其形式化分析中首次系统性指出了授权码拦截与重定向 URI 不严格匹配可能导致的授权码劫持问题，强调了 redirect_uri 参数校验的重要性 [5]。Li 和 Mitchell 在对多个 OAuth 服务提供者的实测分析中发现，许多实现允许模糊匹配（如前缀匹配、通配符域名等），为攻击者注册恶意 URI 留下空间 [25]。此外，Innocenti 揭示了所谓的“redirect URI truncation”攻击路径，即攻击者通过构造视觉相似或被截断的重定向 URI 绕过合法性校验，将授权码重定向到其控制的地址，从而窃取令牌 [21]。

针对此威胁，已有研究提出多种缓解机制。Barth 建议采用白名单精确匹配策略，并在服务端强制 redirect_uri 与 client_id 的唯一绑定关系 [26]。OAuth 2.1 草案也明确规定禁止使用通配符匹配和开放 redirect_uri 注册 [29]。此外，实际部署中（如 Google OAuth 和 GitHub）也逐步引入了 redirect URI pinning、PKCE 强制启用等机制，以降低客户端冒充与 URI 劫持的风险。

尽管如此，实证研究仍不断发现现有服务中存在 bypass 检查逻辑、路径拼接漏洞和 open redirect 的高发问题 [21, 27]，说明该攻击面仍具现实威胁，特别是在移动应用与第三方插件环境中。

2.1.4 客户端冒充

OAuth 实现中的另一个常见漏洞与客户端冒充有关，恶意行为者试图冒充合法客户端以获得对用户资源的未经授权访问 [30, 31]。此漏洞可能导致未经授权的数据访问和隐私泄露，从而突出了 OAuth 系统中健壮的身份验证机制的重要性。在 OAuth 2.0 中，用户授权某个“客户端应用”访问其受保护资源（如用户的电子邮件、联系人等），授权后服务器会将访问令牌发送给该客户端。然而：客户端的身份验证通常依赖于 client_id（和可选的 client_secret）。在某些授权模式（如隐式授权、授权码模式 without PKCE）中，

客户端身份验证弱或被忽略。攻击者可以使用合法客户端的 `client_id` 构造授权链接，欺骗用户点击并登录，从而获得访问令牌（`access token`）并非法访问用户资源。

2.2 SSO 测量工具

早期的对 SSO 系统的安全性研究主要依靠人工方法来评估有限数量的 SPs 和 IdPs。随着 OAuth 和 OpenID Connect 在 Web 应用中的广泛部署，研究者开发了多种自动化工具以评估单点登录（SSO）机制的安全性。

其中，OAuth-Tester [32] 是最早期被广泛使用的 SSO 协议测试框架之一，支持模拟 OAuth 授权流程，并通过手动配置与脚本驱动方式检测重定向 URI 检查绕过、缺失 `state` 参数等实现缺陷。尽管功能灵活，但 OAuth-Tester 主要面向协议级漏洞测试，无法大规模自动化适配现实站点中的多样化 OAuth 实现差异，在动态表单识别、身份绑定等方面存在较大局限。

为提升协议级漏洞发现的系统化能力，Rahat[33] 提出了一种查询驱动的符号执行与路径约束分析方法 Cerberus，用于静态分析服务端 OAuth 实现代码，从源码中提取验证逻辑、分支条件与代码路径，从而识别如客户端伪造、授权码滥用等问题。Cerberus 的优势在于其对服务端代码可见性下的静态高精度漏洞检测能力，但也依赖源码可获取、OAuth 实现可解析的前提，因此难以应用于对闭源系统或大规模 Web 服务的黑盒测量。

相比之下，OAuth-Detector (Singh et al., 2021) 等系统采用浏览器自动化结合 DOM 分析的方式，在无需源码的条件下，自动识别网页中的 OAuth 登录按钮、授权重定向流程与返回参数，并对其安全性与隐私合规性进行判断。该工具在对上千个网站的测量中发现大量 `redirect_uri` 漏洞与权限过度请求现象。然而，OAuth-Detector 的主要限制在于其依赖固定的网页结构模板匹配与有限的协议流程模拟能力，在复杂交互（如多步登录、弹窗授权、混合式 SSO）中识别率有限。

此后，由 Westers 等人于 2023 年提出 SSO-MONITOR[34]，其是第一个开放源码的大规模自动监控 SSO 生态的系统。该工具可以遍历 Tranco 排名前 10k 的网站，自动识别和模拟 Apple、Google、Facebook 登录，并检测 CSRF、open-redirect、参数泄露等安全和隐私问题。SSO-monitor 主要关注于如何识别出存在 SSO 的网站，用于识别 SSO 流程所存在的漏洞的功能仅仅是测试实现，且目前仍然缺少针对国内 SSO 系统的大规模检测报告。

三 课题主要研究内容、预期目标

目前已经有许多关于以 OAuth 为代表的 SSO 系统的安全性分析与漏洞披露，但对于如此大规模的具体实现，仍然存在安全问题亟待解决：针对单点登录系统的安全风险，虽然已经有 OAuth 协议规范给出明确指导，但在实现中仍然因为功能扩展或版本控制等问题存在安全漏洞没有被发现，可能导致大规模的受害者遭到账户接管。此外，目前对 SSO 系统的测量研究，缺乏可用的网站列表数据，缺少可用的开源工具，以及缺少完整的、持续更新的相关测试数据。尤其针对国内 SSO 系统，既没有开源大规模检测工具，又缺乏对国内常用的 OAuth 服务安全性的普遍研究。

3.1 研究内容

本研究致力于提出新的针对以 OAuth 协议为代表的 SSO 系统的新的攻击方法和解决策略，针对上述问题，我们主要研究以下几点内容：

1. 重定向 URI 攻击风险：基于授权码模式和开放重定向漏洞，研究将授权码发送到风险端点且保持未被使用状态的方式。
2. 授权码泄露攻击风险：基于 CSRF 和服务器内存泄漏，针对 PKCE 机制或机密客户端 secret 保护，研究版本降级或绕过安全机制的方式，使得敌手拿到授权码可以登录受害者账户。
3. 针对提出的新的安全风险，研究避免此类风险或保护相关关键参数的方式。
4. SSO 系统安全检测：对国内常用 SSO 站点进行黑盒测试，针对端口转发消息中是否安全实现关键参数，评估存在的安全风险，形成评估报告。

3.2 预期目标

本研究的预期目标如下：

1. 发现使用重定向 URI 攻击使得授权码被发送到危险端点的方式，且据此方法获取的授权码应当保持未被使用状态。
2. 提出基于 PKCE 机制实现漏洞或绕过客户端身份认证安全机制的方式，使用授权码获取访问令牌的方式，最终效果应当是可以以敌手身份登录受害者账户。
3. 针对相关安全风险，提出应对方式，并整理出 OAuth 服务器部署阶段的安全指南。
4. 开发对 SSO 系统进行黑盒测试的测量工具，并对国内常用 SSO 站点进行测试，

形成测试报告并披露安全风险。

四 拟采用的研究方法、技术路线、实验方案及其可行性分析

4.1 研究方法

(1) 理论研究：对 SSO 系统常见风险、黑盒测试、OAuth 客户端-服务器部署的实现方法和关键技术进行详细调研，在充足的理论支持基础上筛选适用的技术方法，形成对本研究所提出的安全风险的理论依据。

(2) 实验研究：部署 OAuth 客户端-服务器，并在其上进行模拟测试，探究不同的安全设置或功能扩展会导致的安全风险。开发 SSO 测量工具，对 SSO 站点进行黑盒测试，对实验结果进行分析和评估。

4.2 技术路线和方案

本研究的技术路线主要由三点组成：研究 OAuth 实现中新的安全漏洞、部署 OAuth 客户端-服务器进行模拟测试、开发 SSO 测量工具。

(1) 研究 OAuth 实现中新的安全漏洞：基于现有 OAuth 实现中暴露出的典型安全隐患，我们拟从“授权码传输路径控制”与“授权码有效性保障”两个关键面出发，系统性挖掘可能存在的新型漏洞。首先，针对重定向 URI 攻击面，我们将结合授权码模式与开放重定向漏洞，研究如何构造未立即使用的授权码泄露路径，使攻击者能够将授权码转发至风险端点而不触发令牌交换，从而规避现有实时检测机制；该路径依赖于客户端对 redirect_uri 参数解析逻辑的不一致或授权服务器对绑定关系的弱约束。其次，针对授权码泄露后的使用阶段，我们将重点分析 PKCE 机制的实际部署差异与客户端 secret 管理薄弱环节，通过版本降级（downgrade）攻击或配置回退路径，尝试复现敌手在获取授权码后可绕过验证约束、成功完成令牌交换的条件。此外，我们还将结合浏览器缓存、Token 回调链分析与服务端状态同步缺陷，构建端到端攻击链，评估在真实部署环境中攻击者可实现账户接管的实际可行性。该技术路线旨在揭示 OAuth 安全机制“形式满足”但“实质绕过”的边界地带，推动对客户端、服务器和标准交互接口的更严格验证机制研究。

(2) 为系统性验证上述攻击思路的可行性与实际影响，我们将构建一套可控的 OAuth 客户端-服务器模拟测试平台，复现授权码传递、验证与令牌交换的全流程，并

在关键交互环节注入异常配置与攻击载荷，进行安全性测试与行为观测。首先，我们将部署标准 OAuth 2.0 授权服务器与可配置的客户端应用，支持授权码模式、PKCE、客户端凭证认证等多种组合。该平台将允许我们灵活修改客户端注册参数（如 redirect URI、client_id/client_secret）及服务器校验策略，以模拟真实世界中配置错误、版本差异或兼容性遗留问题。其次，在客户端与服务器之间构建代理分析层，用于动态拦截并修改授权请求与响应中的关键字段（如 code、state、code_verifier），实现对重定向路径行为、授权码生命周期状态的精细追踪。通过定向构造包含开放重定向、CSRF token 注入、参数污染等特征的请求序列，测试平台可记录服务器响应行为、令牌生成日志。最后，平台将集成安全检测模块，用于评估模拟攻击是否导致授权码泄露、绕过 PKCE 或客户端认证失败检测等问题。通过对比不同配置策略（如启用/禁用 strict redirect 校验、PKCE enforcement）下的系统表现，可量化评估现有防御机制在多种实际部署场景下的鲁棒性与攻击面暴露程度，从而为 OAuth 安全性研究与实现规范提供实验依据。

(3) 为全面评估现实世界中单点登录 (SSO) 站点的协议实现质量与安全风险，我们拟基于 SSO-monitor 开发一套具备黑盒测试能力的 SSO 测量工具链，面向国内主流站点的 OAuth/OpenID Connect 登录机制，自动化执行登录流程、提取协议交互特征，并结合规则引擎识别潜在安全与隐私问题。首先，我们将构建 SSO 行为探测模块，针对目标网站列表逐站点访问页面，识别并点击“使用第三方登录”（如“使用微信登录”）的按钮，触发授权跳转并捕获重定向链、请求参数与响应页面。系统将记录关键交互字段（如 client_id、redirect_uri、scope、response_type 等），并对登录回调数据包进行抓取，以还原 OAuth 流程各阶段的状态。其次，系统将对收集到的参数与行为特征进行语义分析与风险检测。我们将设计若干检测规则与模型，包括：① 判断是否存在开放重定向、未校验 state 参数等常见 CSRF 风险；② 检测是否存在权限过度请求（如请求敏感 scope）、未经用户明确授权的跳转；③ 评估回调 URI 是否使用弱绑定策略（如通配符、路径劫持）；④ 识别多个站点是否共用同一 client_id 以暴露跨站认证风险。

4.3 可行性分析

(1) 理论可行性此技术路线基于已有安全研究（如重定向 URI 漏洞、CSRF 攻击、PKCE 绕过）中已观察到的实际问题，结合 OAuth 协议标准 (RFC 6749、RFC 7636) 中授权码绑定机制的潜在模糊空间，具有明确的攻击建模基础。绕过安全机制（如 code_reuse、PKCE fallback）与信息泄露（如 redirect_uri 未绑定）已被多篇论文实证支持，理论上具

备再现与变种挖掘的空间。OAuth 流程明确划分客户端行为、授权服务器行为、资源服务器行为，具备良好的建模清晰性，适合模块化复现与配置变异测试。

(2) 实验可行性：可利用已有真实配置样例（GitHub、Google 等 OAuth 流程）复现部分场景，也可自建服务端模拟细粒度条件（例如绕过 PKCE 校验的旧版配置）。现有如 Keycloak、Hydra、Dex 等开源 OAuth 实现均可作为服务端部署对象，支持自定义 client 注册与参数配置，方便构造不同测试用例。基于浏览器自动化 + DOM 检测的 SSO 测量工具已有成功先例（如 OAuth-Detector、SSO-Monitor），理论框架成熟，风险识别规则（如 state 缺失、通配 redirect_uri）均为已有验证策略，具备明确的可编程性与验证指标。

五 已有科研基础与所需的科研条件

5.1 已有的科研基础

(1) 理论基础：目前已针对 OAuth 协议流和常见的安全漏洞进行了系统性研究，完全了解 OAuth 协议流程中的关键参数和安全机制，并熟悉与之相关的攻击流程和防护策略。

(2) 实验基础：目前已复现 SSO-monitor，并在原论文数据集上进行前期实验，其次已在 F5 BIG-IP 平台上部署 OAuth 服务器，进行了相关参数的前期设置工作。

5.2 所需的科研条件

对于文献检索，软件所网络可以访问 Web of Science 和 Springer 等文献数据库，相关领域会议上发表的文章可随时获取，满足科研需求。

硬件方面，需配备具备中高性能的测试服务器与浏览器自动化环境，用于运行 SSO-monitor 等工具进行批量 SSO 行为采集与网页交互模拟。服务器应支持 Linux 容器化部署环境（如 Docker、Kubernetes），便于并行运行大规模黑盒扫描任务。

六 研究工作计划与进度安排

阶段	研究任务	时间安排
阶段一	文献调研，梳理 OAuth/SSO 相关安全漏洞模型与防御机制；设计攻击分类与测量指标体系	2025 年 7-2025 年 8 月
阶段二	搭建可控 OAuth 客户端-服务端模拟测试平台，复现典型漏洞场景（如 PKCE 绕过、redirect 劫持）	2025 年 9-2025 年 10 月
阶段三	开发 SSO 黑盒测量工具，部署浏览器自动化采集系统，完成目标站点的 SSO 行为采样与特征提取	2025 年 11 月-2026 年 1 月
阶段四	对实验数据进行分析 and 分类统计，撰写实验报告，总结研究成果，撰写论文与项目总结	2026 年 2-2026 年 4 月

参考文献

- [1] Hardt D. Request for comments: number 6749 The OAuth 2.0 Authorization Framework [M/OL]. RFC Editor, 2012. <https://www.rfc-editor.org/info/rfc6749>. DOI: 10.17487/RFC6749.
- [2] Sakimura N, Bradley J, Jones M, et al. OpenID Connect Core 1.0 incorporating Errata Set 2 [EB/OL]. 2014. https://openid.net/specs/openid-connect-core-1_0.html.
- [3] Hsu C, Lin J C C. Acceptance of blog usage: The roles of technology acceptance, social influence and knowledge sharing motivation [J]. Information & Management, 2008, 45(1): 65-74.
- [4] Bansal C, Bhargavan K, Maffei S. Discovering concrete attacks on website authorization by formal analysis [C/OL]//Proceedings of the 25th IEEE Computer Security Foundations Symposium (CSF). IEEE, 2012: 247-262. <https://doi.org/10.1109/CSF.2012.27>.
- [5] Fett D, Küsters R, Schmitz G. A comprehensive formal security analysis of oauth 2.0 [C/OL]//Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM, 2016: 1204-1215. <https://doi.org/10.1145/2976749.2978385>.

- [6] Andrade R, Yoo S, Tello-Oquendo L, et al. A comprehensive study of the iot cybersecurity in smart cities [J/OL]. IEEE Access, 2020, 8: 228922-228941. <https://doi.org/10.1109/ACCESS.2020.3046442>.
- [7] Sucasas V, Mantas G, Althunibat S, et al. A privacy-enhanced oauth 2.0 based protocol for smart city mobile applications [J/OL]. Computers & Security, 2018, 74: 258-274. <https://doi.org/10.1016/j.cose.2018.01.014>.
- [8] Fett D, Küsters R, Schmitz G. The web sso standard openid connect: In-depth formal security analysis and security guidelines [C/OL]//Proceedings of the 30th IEEE Computer Security Foundations Symposium (CSF). IEEE, 2017: 189-202. <https://doi.org/10.1109/CSF.2017.20>.
- [9] Philippaerts P, Preuveneers D, Joosen W. Oauch: Exploring security compliance in the oauth 2.0 ecosystem [C/OL]//Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (SAC). ACM, 2022: 1844-1853. <https://doi.org/10.1145/3545948.3545955>.
- [10] Munonye K, Martinek P. Machine learning approach to vulnerability detection in oauth 2.0 authentication and authorization flow [J/OL]. International Journal of Information Security, 2021, 21(2): 223-237. <https://doi.org/10.1007/s10207-021-00551-w>.
- [11] Li W, Mitchell C, Chen T. Oauthguard [C/OL]//Proceedings of the 35th Annual Computer Security Applications Conference (ACSAC). ACM, 2019: 444-456. <https://doi.org/10.1145/3338500.3360331>.
- [12] Ferry E, O’Raw J, Curran K. Security evaluation of the oauth 2.0 framework [J/OL]. Information and Computer Security, 2015, 23(1): 73-101. <https://doi.org/10.1108/ICS-12-2013-0089>.
- [13] Sun S, Beznosov K. The devil is in the (implementation) details: An empirical analysis of oauth sso systems [C/OL]//Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS). ACM, 2012: 378-390. <https://doi.org/10.1145/2382196.2382238>.
- [14] Alotaibi A, Mahmmoud A. Enhancing oauth services security by an authentication service with face recognition [C/OL]//Proceedings of the 11th Long Island Systems, Applications and Technology Conference (LISAT). IEEE, 2015. <https://doi.org/10.1109/LISAT.2015.7160208>.
- [15] Dabagh N, Mahmood M. Multilevel database security for android using fast encryption methods [J/OL]. Al-Rafidain Journal of Computer Sciences and Mathematics, 2022, 16(1): 87-96. <https://doi.org/10.33899/csmj.2022.174412>.
- [16] Ismail U, Islam S, Ouedraogo M, et al. A framework for security transparency in cloud computing [J/OL]. Future Internet, 2016, 8(1): 5. <https://doi.org/10.3390/fi8010005>.
- [17] Azizul N, Mohd A, Chandren R, et al. Authentication and authorization design in honeybee computing [J/OL]. International Journal of Advanced Computer Science and Applications (IJACSA), 2019, 10(9). <https://doi.org/10.14569/ijacsa.2019.0100903>. DOI: 10.14569/IJACSA.2019.0100903.
- [18] Microsoft azure account takeover [EB/OL]. 2020. <https://www.cyberark.com/resources/threat-research-blog/blackdirect-microsoft-azure-account-takeover>.
- [19] Facebook security update [EB/OL]. 2018. <https://about.fb.com/news/2018/09/security-update/>.

- [20] Arshad E, Benolli M, Crispo B. Practical attacks on login csrf in oauth [J/OL]. *Computers & Security*, 2022, 121: 102859. <https://doi.org/10.1016/j.cose.2022.102859>.
- [21] Innocenti T, Golinelli M, Onarlioglu K, et al. Oauth 2.0 redirect uri validation falls short, literally [C/OL]//*Proceedings of the 2023 Annual Computer Security Applications Conference (ACSAC ' 23)*. Austin, TX, USA: ACM, 2023: xx–xx. <https://doi.org/10.1145/3627106.3627140>.
- [22] Li W, Mitchell C. Your code is my code: Exploiting a common weakness in oauth 2.0 implementations [M/OL]//Bodden E, Athanasopoulos E, Monroe F. *Lecture Notes in Computer Science: volume 10885 Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2018)*. Springer, 2018: 24-41. https://doi.org/10.1007/978-3-030-03251-7_3.
- [23] Shernan E, Carter H, Tian D, et al. More guidelines than rules: Csrft vulnerabilities from noncompliant oauth 2.0 implementations [C/OL]//*Lecture Notes in Computer Science: volume 9148 Proceedings of the 2015 International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2015)*. Springer, Cham, 2015: 239-260. https://doi.org/10.1007/978-3-319-20550-2_13.
- [24] Bhattacharya S, Najana M, Khanna A, et al. Securing the gatekeeper: Addressing vulnerabilities in oauth implementations for enhanced web security [J/OL]. *International Journal of Global Innovations and Solutions (IJGIS)*, 2024, April 2024. <https://ijgis.pubpub.org/pub/jkavqi25>. DOI: [10.21428/e90189c8.af381673](https://doi.org/10.21428/e90189c8.af381673).
- [25] Li W, Mitchell C J. Security issues in oauth 2.0 sso implementations [C/OL]//*Lecture Notes in Computer Science: volume 8783 International Conference on Information Security (ISC 2014)*. Springer, Cham, 2014: 529-541. https://doi.org/10.1007/978-3-319-13257-0_34.
- [26] Barth A, Jackson C, Mitchell J C. Robust defenses for cross-site request forgery [C/OL]//*Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS ' 08)*. Alexandria, VA, USA: ACM, 2008: 75-88. <https://seclab.stanford.edu/websec/csrf/csrf.pdf>. DOI: [10.1145/1455770.1455784](https://doi.org/10.1145/1455770.1455784).
- [27] Sudhodanan A, Carbone R, Compagna L, et al. Large-scale analysis & detection of authentication cross-site request forgeries [C/OL]//*Proceedings of the 2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. Paris, France: IEEE, 2017: 350-365. <https://doi.org/10.1109/EUROSP.2017.45>.
- [28] Nettleship W, Campbell B, Bradley J, et al. Proof key for code exchange by oauth public clients [M/OL]. *Internet Engineering Task Force*, 2015. <https://www.rfc-editor.org/info/rfc7636>. DOI: [10.17487/RFC7636](https://doi.org/10.17487/RFC7636).
- [29] Parecki A, Hardt D. Oauth 2.1 [EB/OL]. *Internet Engineering Task Force*, 2023. <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1-10>.
- [30] Salt Labs, Salt Security. Oh-auth—abusing oauth to take over millions of accounts [EB/OL]. 2023. <https://salt.security/blog/oh-auth-abusing-oauth-to-take-over-millions-of-accounts>.

- [31] Singh J, Chaudhary N K. Unified singular protocol flow for oauth (uspfo) ecosystem [J/OL]. arXiv preprint arXiv:2301.12496, 2023. <https://arxiv.org/abs/2301.12496>. DOI: 10.48550/arXiv.2301.12496.
- [32] Yang R, Li G, Lau W C, et al. OAuthTester: Model-based security testing of oauth 2.0 implementations [C/OL]//Proceedings of the 2016 ACM Asia Conference on Computer and Communications Security (ASIACCS '16). Xi'an, China: ACM, 2016. <https://doi.org/10.1145/2897845.2897874>.
- [33] Al Rahat T, Feng Y, Tian Y. Cerberus: Query-driven scalable vulnerability detection in oauth service provider implementations [C/OL]//CCS: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22). Los Angeles, CA, USA: ACM, 2022: 1-15. <https://doi.org/10.1145/3548606.3559381>.
- [34] Westers M, Wich T, Jannett L, et al. Sso-monitor: Fully-automatic large-scale landscape, security, and privacy analyses of single sign-on in the wild [J/OL]. arXiv preprint arXiv:2302.01024, 2023. <https://arxiv.org/abs/2302.01024>.